0297. Serialize and Deserialize Binary Tree

Category: Trees | LeetCode: https://leetcode.com/problems/serialize-and-deseria

Problem

- Design serialize(TreeNode) → String and deserialize(String) → TreeNode that round-trip any binary tree.

Intuition

- Preorder with explicit null markers fully determines a tree: when reading back, each "N" says "no child here", so recursion knows exactly when a subtree ends.

Brute Force

- Storing preorder + inorder needs unique values; null markers avoid that restriction.

Optimal Approach

- Serialize: preorder DFS, append val or "N", comma-separated.
- Deserialize: split by commas, read tokens with a pointer; "N" → null; else node, then build left, then right.

Worked Example

```
[1,2,3,null,null,4,5].

serialize -> "1,2,N,N,3,4,N,N,5,N,N"
read 1 -> left: read 2 -> left N, right N
       -> right: read 3 -> left 4 (N,N), right 5 (N,N)
```

Java Solution

```java
public class SerializeDeserializeBinaryTree {
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        write(root, sb);
        return sb.substring(0, sb.length() - 1);
    }

    private void write(TreeNode n, StringBuilder sb) {
        if (n == null) { sb.append("N,"); return; }
        sb.append(n.val).append(',');
        write(n.left, sb);
        write(n.right, sb);
    }

    private String[] tokens;
    private int pos;

    public TreeNode deserialize(String data) {
        tokens = data.split(",");
        pos = 0;
        return read();
    }

    private TreeNode read() {
        String t = tokens[pos++];
        if (t.equals("N")) return null;
        TreeNode n = new TreeNode(Integer.parseInt(t));
        n.left = read();
        n.right = read();
        return n;
    }
}
```

Complexity

- Time: O(n) both ways. Space: O(n).

Pitfalls

- Negative and multi-digit values: always use a delimiter, never single characters.
- Deep skewed trees could hit recursion limits in production; an iterative BFS format avoids that.